

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**

o0o



BÀI TẬP LỚN THỰC TẬP CƠ SỞ

**Tên đề tài: Xây dựng hệ thống tương tác người – máy
bằng cử chỉ tay thời gian thực dựa trên xử lý ảnh và
học máy, kết hợp mô phỏng môi trường 3D**

NHÓM LỚP : 16

Kiều Tiến Đạt

MSSV: B23DCKH020

Nguyễn Khắc Quang Anh

MSSV: B23DCKH004

Giảng viên hướng dẫn: ThS. Hoài Thu

HÀ NỘI, 05/2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc tới ThS. Hoài Thư, giảng viên hướng dẫn đã tận tình chỉ bảo, định hướng và giúp đỡ chúng em trong suốt quá trình thực hiện bài tập lớn môn học Thực tập cơ sở. Những lời khuyên và sự đóng góp chuyên môn quý báu của cô là nền tảng quan trọng giúp chúng em hoàn thiện hệ thống và báo cáo này.

Chúng em cũng xin gửi lời cảm ơn chân thành tới các thầy cô giáo Khoa Công nghệ thông tin 1 – Học viện Công nghệ Bưu chính Viễn thông đã trang bị cho chúng em những kiến thức cơ bản và chuyên ngành hữu ích trong những năm học vừa qua.

Cuối cùng, chúng em xin cảm ơn gia đình, bạn bè đã luôn động viên và tạo điều kiện thuận lợi nhất để chúng em hoàn thành tốt đề tài nghiên cứu này. Dù đã có nhiều cố gắng, bài tập lớn không tránh khỏi những thiếu sót, chúng em rất mong nhận được những ý kiến đóng góp của quý thầy cô để đề tài tiếp tục được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục bài tập lớn.....	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	5
2.1 Tiền xử lý ảnh trực tuyến với OpenCV.....	5
2.2 Trích xuất khớp tay dựa trên học sâu bằng MediaPipe Hands	5
2.3 Nhận dạng cử chỉ tay bằng Học máy và Heuristics	6
2.3.1 Thuật toán luật hình học Heuristics.....	6
2.3.2 Mô hình toán chuẩn hóa tọa độ đặc trưng	8
2.3.3 Nhận dạng cử chỉ tĩnh bằng mạng nơ-ron truyền thẳng (MLP).....	9
2.3.4 Nhận dạng cử chỉ động dựa trên quỹ đạo chuyển động.....	10
2.4 Giao thức truyền dữ liệu UDP thời gian thực	11
2.5 Toán học của bộ lọc thích nghi chống rung One Euro Filter	11
2.6 Các thuật toán hình học mô phỏng 3D trong Unity.....	12
2.6.1 Thuật toán khuếch đại tầm với thích nghi (Reach Remap)	12
2.6.2 Thuật toán xác định trọng tâm và định hướng lòng bàn tay	13
CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ.....	15
3.1 Kiến trúc hệ thống và giao thức truyền dữ liệu.....	15
3.2 Hiện thực hóa module xử lý Python Backend	16
3.3 Lập trình tích hợp tương tác 3D trong Unity	17
3.3.1 Ứng dụng tương tác vật lý 3DHandModel	17
3.3.2 Ứng dụng trò chơi FruitNinja điều khiển bằng cử chỉ.....	19

3.4 Đánh giá kết quả thực nghiệm.....	20
3.4.1 Kết quả huấn luyện và Độ chính xác mô hình học máy	20
3.4.2 Hiệu năng thời gian thực và Độ trễ hệ thống	24
3.4.3 Đánh giá hiệu quả làm mịn của bộ lọc One Euro Filter	24
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	26
4.1 Kết luận	26
4.2 Hướng phát triển.....	27
TÀI LIỆU THAM KHẢO.....	29

DANH MỤC HÌNH VẼ

Hình 2.1	Sơ đồ cấu trúc 21 điểm landmark khớp tay trong MediaPipe . . .	6
Hình 3.1	Sơ đồ khối kiến trúc hệ thống tương tác cử chỉ tay thời gian thực	15
Hình 3.2	Giao diện mô phỏng xương bàn tay 3D và hành động cầm nắm vật thể ảo trong Unity	19
Hình 3.3	Giao diện trò chơi FruitNinja điều khiển bằng cử chỉ ngón trỏ (Pointer)	20
Hình 3.4	Ma trận nhầm lẫn của mô hình phân loại cử chỉ tĩnh bằng mạng MLP	21
Hình 3.5	Ma trận nhầm lẫn của thuật toán phân loại cử chỉ tĩnh bằng luật Heuristics	22
Hình 3.6	Ma trận nhầm lẫn của mô hình phân loại cử chỉ động	23

DANH MỤC BẢNG BIỂU

Bảng 3.1	Báo cáo kết quả phân loại chi tiết (Classification Report) của mô hình cử chỉ tĩnh MLP	21
Bảng 3.2	Báo cáo kết quả phân loại chi tiết (Classification Report) của thuật toán hình học Heuristics	22
Bảng 3.3	Bảng so sánh hiệu năng giữa mô hình học máy MLP và thuật toán hình học Heuristics	22
Bảng 3.4	Báo cáo kết quả phân loại chi tiết (Classification Report) của mô hình cử chỉ động	24

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CSV	Tập giá trị phân tách bằng dấu phẩy (Comma-Separated Values)
FPS	Số khung hình hiển thị trên mỗi giây (Frames Per Second)
HCI	Tương tác người - máy (Human - Computer Interaction)
LSTM	Mạng bộ nhớ ngắn hạn dài (Long Short-Term Memory)
MLP	Mạng nơ-ron truyền thẳng nhiều lớp (Multi-Layer Perceptron)
NUI	Giao diện người dùng tự nhiên (Natural User Interface)
TFLite	Phiên bản tối ưu hóa TensorFlow cho thiết bị di động và nhúng (TensorFlow Lite)
UDP	Giao thức gói người dùng (User Datagram Protocol)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Tổng quan chương

Chương 1 giới thiệu tổng quan về đề tài nghiên cứu và xây dựng hệ thống tương tác người - máy bằng cử chỉ tay thời gian thực. Nội dung chương bao gồm bốn phần chính: (i) Đặt vấn đề nghiên cứu, phân tích tính cấp thiết của tương tác tự nhiên trong không gian ba chiều và các hạn chế của thiết bị ngoại vi truyền thống; (ii) Xác định mục tiêu nghiên cứu và phạm vi ứng dụng của đề tài; (iii) Đề xuất định hướng giải pháp kỹ thuật dựa trên công nghệ học máy kết hợp mô phỏng 3D; và (iv) Trình bày bố cục chung của báo cáo bài tập lớn.

1.1 Đặt vấn đề

Tương tác người - máy (HCI - Human-Computer Interaction) đã trải qua nhiều giai đoạn phát triển từ các giao diện dòng lệnh (CLI), giao diện người dùng đồ họa 2D (GUI) đến các giao diện người dùng tự nhiên (NUI - Natural User Interface). Trong các môi trường làm việc văn phòng truyền thống, chuột và bàn phím là các thiết bị nhập liệu tối ưu. Tuy nhiên, sự bùng nổ của các ứng dụng đồ họa 3D phức tạp, các môi trường thực tế ảo (VR), thực tế tăng cường (AR) và các trò chơi mô phỏng vật lý đã đòi hỏi một phương thức tương tác trực quan và tự nhiên hơn. Các thiết bị 2D truyền thống bộc lộ những hạn chế lớn về số bậc tự do nhập liệu, khiến người dùng gặp khó khăn trong việc điều khiển, định vị và thao tác trực tiếp với các đối tượng trong không gian ba chiều.

Để giải quyết vấn đề này, việc nghiên cứu các giao diện người dùng tự nhiên dựa trên cử chỉ bàn tay (hand gesture interaction) là hướng đi đầy triển vọng [1]. Các chức năng điều khiển dựa trên đầu vào là bàn tay con người từ đó tạo ra nhiều tư thế và chuyển động phục vụ cho các chức năng trong game. Tương tác cử chỉ tay cho phép người dùng điều khiển hệ thống một cách trực tiếp mà không cần tiếp xúc vật lý, mang lại trải nghiệm nhập liệu tự nhiên và trực quan.

Mặc dù các cảm biến chiều sâu chuyên dụng như Leap Motion, Kinect hay các thiết bị găng tay dữ liệu (data gloves) có thể cung cấp tọa độ bàn tay với độ chính xác cao, nhưng chi phí trang bị đắt đỏ và tính tương thích phần cứng hạn chế đã ngăn cản sự phổ biến của chúng đến người dùng phổ thông. Vì vậy, bài toán đặt ra là làm thế nào để xây dựng một hệ thống nhận dạng cử chỉ bàn tay thời gian thực chỉ sử dụng camera RGB (webcam) thông thường có sẵn trên hầu hết các máy tính hiện nay. Giải pháp này đòi hỏi sự kết hợp mạnh mẽ giữa các công nghệ xử lý ảnh trực tuyến, trích xuất điểm mốc bàn tay dựa trên mô hình học sâu và phân loại cử chỉ bằng các thuật toán học máy tối ưu nhằm đạt được tốc độ xử lý cao, độ trễ thấp

và độ chính xác phù hợp trong điều kiện phần cứng máy tính cá nhân phổ thông.

1.2 Mục tiêu và phạm vi đề tài

Đề tài hướng tới việc nghiên cứu, thiết kế và phát triển một hệ thống tương tác người - máy bằng cử chỉ tay thời gian thực, kết hợp giữa thuật toán học máy phân loại cử chỉ trên luồng ảnh webcam và môi trường mô phỏng đồ họa 3D tương tác.

Các mục tiêu cụ thể được đặt ra bao gồm:

1. Nghiên cứu phương pháp trích xuất đặc trưng hình học bàn tay thời gian thực, ứng dụng framework MediaPipe Hands của Google để dò tìm bàn tay và định vị chính xác tọa độ không gian 3D của 21 điểm khớp mốc chính.
2. Xây dựng thuật toán chuẩn hóa dữ liệu đầu vào để loại bỏ các ảnh hưởng do khoảng cách bàn tay tới camera, vị trí của bàn tay trong khung hình, đảm bảo tính bất biến về vị trí và tỉ lệ của đặc trưng hình học.
3. Thiết kế và huấn luyện hai mô hình phân loại cử chỉ: (i) bộ phân loại cử chỉ tĩnh (6 lớp cử chỉ chính bao gồm Open, Close, Pointer, OK, Peace, ThumbsUp) bằng mạng nơ-ron truyền thẳng (MLP) tối ưu hóa sang định dạng TensorFlow Lite, và (ii) bộ phân loại cử chỉ động (10 lớp cử chỉ di chuyển bao gồm các động tác vuốt Swipe Left/Right/Up/Down, vẽ hình tròn Circle, tam giác Triangle, ký tự Z/S, vô cực Infinity) dựa trên chuỗi thời gian quỹ đạo của ngón trỏ. Đồng thời tiến hành đánh giá đối chứng hiệu năng giữa thuật toán luật hình học Heuristics (rule-based) và mô hình mạng nơ-ron MLP để làm nổi bật ưu thế của từng phương pháp.
4. Lập trình tích hợp tương tác trong môi trường 3D bằng engine Unity thông qua kết nối mạng truyền thông điệp UDP cục bộ độ trễ thấp để điều khiển hai ứng dụng thực tế: (i) ứng dụng mô phỏng xương tay ảo 3DHandModel tích hợp cơ chế cầm nắm, giữ và ném vật lý dựa trên các cử chỉ nhận dạng tương ứng; và (ii) ứng dụng game tương tác FruitNinja điều khiển lưỡi dao chém hoa quả bằng ngón trỏ, tích hợp giải thuật lọc nhiễu thích nghi One Euro Filter trên vị trí lưỡi dao để loại bỏ rung giật tọa độ chuyển động tay.

Phạm vi nghiên cứu của đề tài tập trung vào việc xử lý chuyển động bàn tay đơn lẻ từ một luồng camera RGB duy nhất đặt trước người dùng. Nghiên cứu chủ yếu giải quyết các vấn đề kỹ thuật liên quan đến độ ổn định của dữ liệu chuyển động, cải thiện tốc độ xử lý của pipeline AI nhằm hướng tới tần số xử lý khoảng trên 30 khung hình/giây (FPS) và giảm độ trễ phản hồi hệ thống để tạo cảm giác tương tác bám đuổi tốt hơn.

1.3 Định hướng giải pháp

Giải pháp kỹ thuật được đề xuất sử dụng kiến trúc phân tách Client-Server chạy song song trên cùng một máy tính:

1. **Phía Server (Python AI Backend):** Nhận nhiệm vụ thu nhận hình ảnh từ webcam ở tần số cao bằng thư viện OpenCV, chạy mô hình học sâu MediaPipe để dò tìm và xuất ra danh sách 21 điểm mốc tọa độ 3D của bàn tay. Các tọa độ này được đưa qua bộ tiền xử lý chuẩn hóa đặc trưng. Server sử dụng các mô hình TensorFlow Lite (.tflite) để thực hiện suy luận cử chỉ tĩnh và cử chỉ động ở phía Python. Trong phiên bản hiện tại, dữ liệu truyền sang Unity qua UDP gồm tọa độ 21 điểm khớp tay, nhãn cử chỉ tĩnh và kích thước khung hình camera.
2. **Phía Client (Unity 3D Frontend):** Lập trình một luồng nhận dữ liệu UDP độc lập để liên tục cập nhật vị trí cho mô hình bàn tay ảo hoặc lưỡi dao. Tọa độ của ngón tay trở điều khiển lưỡi dao trong game Fruit Ninja được lọc nhiễu tần số cao thông qua thuật toán One Euro Filter trước khi ánh xạ vào hệ tọa độ màn hình game. Dựa trên trạng thái cử chỉ nhận được (ví dụ nhãn "Close" biểu thị hành động nắm tay, nhãn "Pointer" biểu thị hành động chỉ ngón trỏ), hệ thống sẽ điều khiển logic cầm nắm vật thể 3D vật lý hoặc lưỡi dao chém quả.

Định hướng giải pháp này giúp phân tách rõ ràng nhiệm vụ tính toán AI và kết xuất đồ họa, giải phóng tài nguyên cho hệ thống, đồng thời sử dụng kết nối UDP giúp giảm chi phí truyền nhận giữa Python và Unity trong thử nghiệm cục bộ.

1.4 Bố cục bài tập lớn

Báo cáo bài tập lớn Thực tập cơ sở này được tổ chức thành 4 chương chính như sau:

Chương 2 trình bày cơ sở lý thuyết liên quan đến hệ thống, bao gồm nguyên lý hoạt động của OpenCV trong xử lý ảnh số thời gian thực, kiến trúc trích xuất đặc trưng khớp tay của thư viện MediaPipe, phương pháp toán học chuẩn hóa tọa độ đặc trưng, lý thuyết mạng nơ-ron nhân tạo truyền thẳng (MLP) cho phân loại cử chỉ tĩnh và động, giao thức truyền thông UDP thời gian thực và giải thuật lọc thích nghi chống rung One Euro Filter.

Chương 3 tập trung mô tả chi tiết thiết kế hệ thống, kiến trúc các module phần mềm, quá trình hiện thực hóa các lớp xử lý dữ liệu và thuật toán điều khiển trong Unity (bao gồm kịch bản đồng bộ xương tay ảo, cơ chế tương tác cầm nắm vật lý và lập trình trò chơi tương tác Fruit Ninja). Đồng thời, chương này trình bày các

kết quả đánh giá thực nghiệm về độ chính xác phân loại cử chỉ, tốc độ xử lý FPS và độ trễ phản hồi của hệ thống.

Chương 4 đưa ra những đánh giá tổng kết về các đóng góp chính của đề tài, phân tích những hạn chế kỹ thuật hiện tại và đề xuất định hướng phát triển, cải tiến hệ thống trong tương lai nhằm nâng cao độ chính xác nhận dạng và mở rộng không gian tương tác.

Kết chương

Chương 1 đã phác thảo bức tranh tổng quan về đề tài tương tác cử chỉ bàn tay 3D thời gian thực, làm rõ tính cấp thiết, mục tiêu nghiên cứu và giải pháp thiết kế hệ thống Client-Server phân tán. Để có cơ sở lý thuyết vững chắc cho việc cài đặt và hiện thực hóa các module xử lý này, Chương 2 tiếp theo sẽ đi sâu nghiên cứu các thuật toán trích xuất đặc trưng hình học, nhận dạng cử chỉ học máy và bộ lọc chống rung thích nghi.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Tổng quan chương

Chương 1 đã phân thảo các mục tiêu và giải pháp thiết kế tổng thể của hệ thống tương tác cử chỉ tay 3D thời gian thực. Để xây dựng hệ thống hoạt động ổn định và có độ chính xác cao trên các camera phổ thông, việc nghiên cứu các thuật toán nền tảng đóng vai trò quyết định. Chương 2 này sẽ trình bày chi tiết cơ sở lý thuyết khoa học của các thành phần trong hệ thống, bao gồm: (i) tiền xử lý luồng video trực tuyến với OpenCV; (ii) nguyên lý trích xuất 21 điểm mốc khớp bàn tay bằng mô hình học sâu MediaPipe Hands; (iii) các thuật toán chuẩn hóa tọa độ hình học và mô hình mạng nơ-ron truyền thẳng (MLP) cho phân loại cử chỉ tĩnh và động; (iv) cơ chế truyền nhận dữ liệu qua socket UDP; và (v) toán học của giải thuật lọc thích nghi chống rung One Euro Filter.

2.1 Tiền xử lý ảnh trực tuyến với OpenCV

Thư viện OpenCV (Open Source Computer Vision Library) [2] đóng vai trò là module thu nhận dữ liệu đầu vào. OpenCV giao tiếp trực tiếp với webcam thông qua lớp `cv2.VideoCapture` để nắm bắt các khung hình liên tiếp ở tần số cao (30 khung hình/giây). Các khung hình thu được từ camera thường ở định dạng màu sắc BGR (Blue-Green-Red) mặc định.

Trước khi đưa vào mô hình học sâu để xử lý, ảnh thô cần trải qua các bước tiền xử lý biến đổi hình học và không gian màu sắc: (i) lật ảnh theo chiều ngang (Horizontal Flip) bằng thuật toán biến đổi hình học trong OpenCV để lật khung hình theo trục dọc, đảm bảo chuyển động tự nhiên như nhìn gương; (ii) chuyển đổi không gian màu BGR sang RGB vì MediaPipe thường xử lý ảnh đầu vào theo định dạng RGB; và (iii) giảm độ phân giải khung hình (Downscaling) xuống mức HD (960×540) bằng bộ lọc nội suy tuyến tính để giảm tải dữ liệu pixel, hỗ trợ duy trì tốc độ xử lý khoảng trên 30 FPS trong điều kiện thử nghiệm, đồng thời vẫn giữ chất lượng định vị khớp tay ở mức chấp nhận được.

2.2 Trích xuất khớp tay dựa trên học sâu bằng MediaPipe Hands

Sau khi tiền xử lý ảnh đầu vào, hệ thống trích xuất đặc trưng khớp bàn tay bằng thư viện MediaPipe Hands của Google [3]. Đây là giải pháp học sâu tích hợp sẵn, bao gồm hai mô hình mạng nơ-ron hoạt động nối tiếp nhau:

1. **Mạng phát hiện bàn tay (Hand Detector Model):** Sử dụng kiến trúc Single Shot Detector (SSD) được tối ưu hóa cho các thiết bị di động để nhanh chóng phát hiện vùng chứa bàn tay trong toàn bộ khung hình ảnh và xuất ra một

khung bao giới hạn (bounding box).

2. **Điểm mốc bàn tay (Hand Landmark Model):** Chạy trên vùng ảnh bàn tay đã được cắt ra từ mô hình phát hiện để dự đoán tọa độ 3D tương đối của 21 điểm mốc khớp bàn tay (landmarks). Sơ đồ 21 điểm mốc này được biểu diễn trong Hình 2.1.



Hình 2.1: Sơ đồ cấu trúc 21 điểm landmark khớp tay trong MediaPipe

Dữ liệu đầu ra của MediaPipe cho mỗi điểm mốc i ($i \in [0, 20]$) là một bộ ba tọa độ không gian (x_i, y_i, z_i) , trong đó: x_i và y_i biểu thị tọa độ ngang và dọc của điểm mốc trên khung hình, đã được chuẩn hóa về khoảng $[0.0, 1.0]$ bằng cách chia cho chiều rộng và chiều cao khung hình; còn z_i biểu thị độ sâu tương đối của điểm mốc đó so với điểm cổ tay (Wrist - điểm số 0), với giá trị z_i càng nhỏ biểu thị điểm khớp càng nằm gần camera hơn.

2.3 Nhận dạng cử chỉ tay bằng Học máy và Heuristics

Hệ thống sử dụng đồng thời hai phương pháp nhận dạng cử chỉ bàn tay nhằm cân bằng giữa tốc độ xử lý và khả năng nhận dạng các mẫu cử chỉ phức tạp.

2.3.1 Thuật toán luật hình học Heuristics

Phương pháp nhận dạng dựa trên thuật toán luật hình học Heuristics (rule-based) phân tích trực tiếp cấu trúc bàn tay thông qua trạng thái co hay duỗi của từng ngón tay để đưa ra nhãn cử chỉ tĩnh tức thời mà không yêu cầu tài nguyên tính toán lớn hay dữ liệu huấn luyện trước.

Đối với bốn ngón tay dài bao gồm ngón trỏ, ngón giữa, ngón áp út và ngón út, hệ thống xác định trạng thái co/duỗi dựa trên khoảng cách hình học từ khớp cổ tay đến đầu ngón so với khoảng cách đến khớp giữa của ngón đó. Cụ thể, mỗi ngón tay $f \in \{\text{trỏ, giữa, áp út, út}\}$ có điểm đầu ngón là Tip_f (tương ứng với các điểm landmark 8, 12, 16, 20) và điểm khớp giữa PIP là Pip_f (tương ứng với các điểm landmark 6, 10, 14, 18). Để triệt tiêu hoàn toàn sai số độ sâu trục z từ một camera RGB duy nhất, khoảng cách Euclidean 2D trên mặt phẳng ảnh (x, y) được sử dụng theo công thức (2.1):

$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2} \quad (2.1)$$

Trạng thái co/duỗi của ngón tay dài f được xác định theo quy tắc trong phương trình (2.2):

$$State(f) = \begin{cases} 1 \text{ (duỗi)} & \text{nếu } d(P_{Tip_f}, P_0) > d(P_{Pip_f}, P_0) \\ 0 \text{ (co)} & \text{nếu } d(P_{Tip_f}, P_0) \leq d(P_{Pip_f}, P_0) \end{cases} \quad (2.2)$$

Trong đó P_0 là tọa độ điểm cổ tay (Wrist - landmark 0).

Đối với ngón cái (Thumb), do cấu trúc giải phẫu chuyển động khác biệt, hệ thống sử dụng một khoảng cách tham chiếu tính từ gốc ngón trỏ đến gốc ngón út nhằm loại bỏ sai số co giãn khi tay tiến lại gần hoặc ra xa camera. Trạng thái ngón cái được đánh giá bằng cách so sánh khoảng cách giữa đầu ngón cái P_4 (landmark 4) với gốc ngón út P_{17} (landmark 17) so với khoảng cách tham chiếu giữa gốc ngón trỏ P_5 (landmark 5) và gốc ngón út P_{17} theo phương trình (2.3):

$$State(\text{cái}) = \begin{cases} 1 \text{ (duỗi)} & \text{nếu } d(P_4, P_{17}) > 1.2 \times d(P_5, P_{17}) \\ 0 \text{ (co)} & \text{nếu } d(P_4, P_{17}) \leq 1.2 \times d(P_5, P_{17}) \end{cases} \quad (2.3)$$

Tập hợp trạng thái của 5 ngón tay được biểu diễn dưới dạng một vectơ nhị phân S theo công thức (2.4):

$$S = [State(\text{cái}), State(\text{trỏ}), State(\text{giữa}), State(\text{áp út}), State(\text{út})] \quad (2.4)$$

Dựa vào vectơ này, hệ thống ánh xạ ra các cử chỉ tĩnh cơ bản theo các luật sau:

- **Cử chỉ Open (Bàn tay mở):** Tất cả các ngón tay đều duỗi, tương ứng với vectơ trạng thái $S = [1, 1, 1, 1, 1]$.
- **Cử chỉ Close (Nắm đấm):** Tất cả các ngón tay đều co, tương ứng với vectơ trạng thái $S = [0, 0, 0, 0, 0]$.
- **Cử chỉ Pointer (Chỉ tay):** Chỉ ngón trỏ duỗi, các ngón còn lại co, tương ứng với vectơ trạng thái $S = [0, 1, 0, 0, 0]$.
- **Cử chỉ Peace (Chào chiến thắng):** Ngón trỏ và ngón giữa duỗi, các ngón khác co, tương ứng với vectơ trạng thái $S = [0, 1, 1, 0, 0]$.
- **Cử chỉ ThumbsUp (Like):** Chỉ ngón cái duỗi, các ngón khác co, tương ứng với vectơ trạng thái $S = [1, 0, 0, 0, 0]$.

Riêng đối với cử chỉ phức tạp **OK**, do ngón trỏ uốn cong để chạm vào ngón cái trong khi các ngón còn lại duỗi, vectơ trạng thái nhị phân thô dễ bị nhầm lẫn. Để tăng độ chính xác, hệ thống tính toán khoảng cách chuẩn hóa độc lập. Đầu tiên,

chiều dài lòng bàn tay tham chiếu được xác định qua khoảng cách giữa cổ tay P_0 và gốc ngón giữa P_9 : $palm_len = d(P_0, P_9)$. Cử chỉ OK được kích hoạt khi ngón giữa duỗi ($State(giữa) = 1$) đồng thời khoảng cách giữa đầu ngón cái P_4 và đầu ngón trỏ P_8 nhỏ hơn ngưỡng tỉ lệ 40% chiều dài lòng bàn tay, được xác định qua điều kiện (2.5):

$$Class = OK \quad \text{nếu} \quad State(giữa) == 1 \quad \text{và} \quad d(P_4, P_8) < 0.4 \times d(P_0, P_9) \quad (2.5)$$

Để triệt tiêu hiện tượng nhấp nháy nhãn (label flickering) khi bàn tay nằm ở ranh giới giữa các trạng thái co và duỗi, nhãn dự đoán thô từ heuristics được đưa qua một hàng đợi FIFO có kích thước cửa sổ lọc mượt $RULE_SMOOTH_WINDOW = 5$ khung hình liên tiếp. Nhãn heuristics cuối cùng được xác định thông qua nhãn có tần suất xuất hiện nhiều nhất (Majority Voting) trong cửa sổ này, giúp duy trì tín hiệu điều khiển ổn định truyền sang Unity.

2.3.2 Mô hình toán chuẩn hóa tọa độ đặc trưng

Khi sử dụng mô hình học máy để nhận diện nhiều cử chỉ phức tạp, tọa độ thô từ MediaPipe phụ thuộc lớn vào vị trí của bàn tay trong ảnh và khoảng cách của tay tới camera. Để đảm bảo mô hình nhận dạng hoạt động ổn định bất kể tay người dùng ở xa hay gần camera, lệch trái hay lệch phải khung hình, thuật toán chuẩn hóa tọa độ hình học bàn tay được thực hiện như sau:

Bước 1: Tịnh tiến (translation) toàn bộ tọa độ các điểm mốc bàn tay $P_i = (x_i, y_i)$ về gốc tọa độ bằng cách lấy tọa độ điểm cổ tay $P_0 = (x_0, y_0)$ làm điểm mốc chuẩn theo phương trình (2.6):

$$P'_i = P_i - P_0 = (x_i - x_0, y_i - y_0), \quad \forall i \in [0, 20] \quad (2.6)$$

Bước 2: Chuyển đổi danh sách 21 điểm mốc P'_i sau tịnh tiến thành một mảng một chiều V gồm 42 phần tử dạng $[x'_0, y'_0, x'_1, y'_1, \dots, x'_{20}, y'_{20}]$. Tìm giá trị tuyệt đối lớn nhất trong mảng một chiều này để xác định hệ số tỉ lệ lớn nhất d_{max} theo công thức (2.7):

$$d_{max} = \max_{j \in [0, 41]} |V_j| \quad (2.7)$$

Bước 3: Chia toàn bộ các phần tử trong mảng một chiều cho hệ số tỉ lệ d_{max} để thu được mảng tọa độ chuẩn hóa đặc trưng V'' theo phương trình (2.8):

$$V''_j = \frac{V_j}{d_{max}}, \quad \forall j \in [0, 41] \quad (2.8)$$

Mảng đặc trưng V'' gồm 42 phần tử luôn nằm trong khoảng giới hạn $[-1.0, 1.0]$. Phép toán này giảm đáng kể ảnh hưởng của vị trí và kích thước biểu kiến của bàn tay đến camera nhờ việc chuẩn hóa kích thước bàn tay về một tỷ lệ đơn vị thống nhất, tạo điều kiện thuận lợi cho việc huấn luyện mô hình phân loại.

2.3.3 Nhận dạng cử chỉ tĩnh bằng mạng nơ-ron truyền thẳng (MLP)

Để phân loại 6 cử chỉ tĩnh chính bao gồm Open, Close, Pointer, OK, Peace và ThumbsUp, hệ thống sử dụng mạng nơ-ron nhân tạo truyền thẳng nhiều lớp MLP (Multi-Layer Perceptron). Kiến trúc mạng được thiết kế tối ưu với cấu trúc liên kết đầy đủ (Fully Connected) bao gồm một lớp đầu vào nhận vectơ đặc trưng chuẩn hóa V'' gồm 42 chiều, hai lớp ẩn trung gian để học các biểu diễn phi tuyến phức tạp, và một lớp đầu ra dự đoán. Lớp ẩn thứ nhất có 64 nơ-ron được tích hợp kỹ thuật Dropout với tỉ lệ loại bỏ là 0.3 để chống hiện tượng quá khớp (overfitting), và lớp ẩn thứ hai có 32 nơ-ron.

Các lớp ẩn trung gian sử dụng hàm kích hoạt phi tuyến ReLU (Rectified Linear Unit) được định nghĩa qua phương trình (2.9) nhằm tăng tốc độ hội tụ và tránh hiện tượng triệt tiêu đạo hàm (vanishing gradient):

$$f(x) = \max(0, x) \quad (2.9)$$

Lớp đầu ra của mạng có 6 nơ-ron tương ứng với 6 lớp cử chỉ tĩnh cần phân loại. Hệ thống áp dụng hàm kích hoạt Softmax ở lớp cuối cùng để chuyển đổi các giá trị đầu ra thô (logits) z_c thành phân phối xác suất dự đoán P cho từng lớp cử chỉ theo phương trình (2.10):

$$P(class = c|V'') = \frac{e^{z_c}}{\sum_{k=1}^6 e^{z_k}} \quad (2.10)$$

Mô hình được huấn luyện bằng cách tối ưu hóa hàm mất mát entropy chéo đa lớp dạng thưa (Sparse Categorical Cross-Entropy Loss) được mô tả trong phương trình (2.11) sử dụng bộ tối ưu hóa Adam:

$$\mathcal{L} = -\log(P(class = y_{true}|V'')) \quad (2.11)$$

Trong đó y_{true} là chỉ số nhãn thực tế của lớp cử chỉ. Sau khi huấn luyện và lựa chọn bộ trọng số có kết quả tốt trên tập kiểm chứng, mô hình được xuất và chuyển đổi sang định dạng TensorFlow Lite (.tflite) [4]. Định dạng TensorFlow Lite (.tflite) giúp mô hình thuận tiện hơn cho quá trình triển khai và suy luận trong các ứng dụng thời gian thực, đồng thời có thể hỗ trợ các kỹ thuật tối ưu hóa dung lượng và tốc độ thực thi trên CPU phổ thông.

2.3.4 Nhận dạng cử chỉ động dựa trên quỹ đạo chuyển động

Nhận dạng cử chỉ động bao gồm các động tác vuốt tay Swipe, vẽ các hình hình học Circle, Triangle, Z-shape đòi hỏi hệ thống phải nắm bắt được thông tin chuyển động theo trục thời gian liên tiếp. Để thực hiện điều này, hệ thống lưu trữ lịch sử tọa độ không gian của điểm đầu ngón tay trở (landmark số 8) trong một hàng đợi FIFO (First-In, First-Out) có kích thước cố định $N = 32$ khung hình gần nhất.

Để mô hình phân loại cử chỉ động có khả năng hoạt động độc lập với vị trí người dùng vẽ cử chỉ trên màn hình và tốc độ vẽ nhanh hay chậm, chuỗi hành trình tọa độ thô thu được $[(x_0, y_0), (x_1, y_1), \dots, (x_{31}, y_{31})]$ cần được chuẩn hóa. Hệ thống tịnh tiến toàn bộ quỹ đạo về điểm khởi đầu của chuỗi $(x_{start}, y_{start}) = (x_0, y_0)$, đồng thời chia cho kích thước độ phân giải thực tế của khung hình camera (W, H) để khử sự phụ thuộc tỉ lệ theo phương trình (2.12):

$$x_k^{norm} = \frac{x_k - x_0}{W}, \quad y_k^{norm} = \frac{y_k - y_0}{H}, \quad \forall k \in [0, 31] \quad (2.12)$$

Mảng phẳng một chiều gồm 64 phần tử tọa độ chuẩn hóa (biểu diễn dưới dạng $[x_0^{norm}, y_0^{norm}, \dots, x_{31}^{norm}, y_{31}^{norm}]$) đại diện cho hình dáng hình học thuần túy của quỹ đạo vẽ. Mảng đặc trưng này được đưa vào mô hình học máy thứ hai là một mạng MLP chuyên biệt dành cho cử chỉ động. Mạng MLP này có lớp đầu vào 64 chiều, được đưa qua một lớp Dropout với tỉ lệ 0.2, tiếp nối bởi lớp ẩn thứ nhất gồm 128 nơ-ron tích hợp Dropout tỉ lệ 0.4, lớp ẩn thứ hai gồm 64 nơ-ron sử dụng kích hoạt ReLU, và lớp đầu ra sử dụng hàm Softmax gồm 10 nơ-ron tương ứng với 10 lớp cử chỉ động.

Sau khi mạng MLP xuất ra kết quả dự đoán thô cho từng khung hình, nhằm triệt tiêu hiện tượng nhảy nhãn liên tục (label flickering) gây ra bởi nhiễu số từ camera hoặc sai số phân loại tức thời, hệ thống áp dụng cơ chế lọc mượt phi tham số dựa trên biểu quyết đa số (Majority Voting) trong một cửa sổ thời gian trượt. Kỹ thuật này được hiện thực hóa thông qua cấu trúc hàng đợi hai đầu deque với độ dài cố định $M = 32$ phần tử để liên tục tích lũy các nhãn dự đoán thô L_i của các khung hình gần nhất. Nhãn cử chỉ động đầu ra cuối cùng $Class_{final}$ được xác định bằng cách thống kê tần suất và lựa chọn nhãn xuất hiện nhiều nhất trong hàng đợi trượt theo phương trình (2.13):

$$Class_{final} = \arg \max_{c \in C} \sum_{i=1}^M \mathbb{I}(L_i = c) \quad (2.13)$$

Trong đó: C là tập hợp 10 lớp cử chỉ động; L_i là nhãn dự đoán thô ở khung hình

thứ i trong cửa sổ trượt; và $\mathbb{I}(\cdot)$ là hàm chỉ thị nhận giá trị bằng 1 nếu biểu thức bên trong đúng, nhận giá trị bằng 0 nếu biểu thức sai. Giải pháp làm mượt này giúp ổn định kết quả nhận dạng ở phía Python, tạo cơ sở để tích hợp vào Unity trong các phiên bản tiếp theo, giúp cân bằng hiệu quả giữa tốc độ phản hồi thời gian thực và sự chính xác bền vững của hệ thống.

2.4 Giao thức truyền dữ liệu UDP thời gian thực

Để truyền dữ liệu tọa độ khớp tay và kết quả nhận dạng cử chỉ từ module Python sang môi trường mô phỏng Unity 3D [5], hệ thống sử dụng giao thức truyền tin qua UDP (User Datagram Protocol) [6].

UDP là giao thức mạng phi kết nối hoạt động ở tầng vận chuyển. So với giao thức TCP đòi hỏi thiết lập kết nối (bắt tay 3 bước) và cơ chế truyền lại khi mất gói tin (gây ra hiện tượng trễ tích lũy cực kỳ nguy hiểm cho các ứng dụng tương tác thời gian thực), UDP cho phép gửi các gói tin một cách độc lập và trực tiếp. Khi dữ liệu được cập nhật liên tục từ camera, việc mất một số gói tin đơn lẻ thường ít ảnh hưởng đến trạng thái hiển thị hơn so với các ứng dụng yêu cầu truyền dữ liệu toàn vẹn, vì gói tin của khung hình tiếp theo có thể nhanh chóng cập nhật trạng thái mới. Do đó, UDP phù hợp với bài toán truyền dữ liệu điều khiển thời gian thực trong phạm vi thử nghiệm cục bộ của đề tài.

2.5 Toán học của bộ lọc thích nghi chống rung One Euro Filter

Tọa độ trích xuất từ camera thường có độ nhiễu tần số cao do rung tay vật lý hoặc sai số từ mô hình MediaPipe. Hiện tượng này làm cho lưỡi dao chém hoa quả trong game Fruit Ninja trong Unity bị rung nhấp nháy liên tục (jitter). Để giảm hiện tượng này, bộ lọc thông thấp thích nghi One Euro Filter [7] được áp dụng trên tọa độ điều khiển của lưỡi dao phía Unity.

One Euro Filter hoạt động dựa trên bộ lọc thông thấp bậc nhất với tần số cắt f_c thay đổi động theo vận tốc của tín hiệu. Cho một tín hiệu đầu vào x_k tại thời điểm t_k , khoảng thời gian trôi qua là $dt = t_k - t_{k-1}$. Các bước tính toán được thực hiện như sau:

Bước 1: Tính đạo hàm rời rạc của tín hiệu đầu vào để ước lượng vận tốc thô dx_k theo công thức (2.14):

$$dx_k = \frac{x_k - \hat{x}_{k-1}}{dt} \quad (2.14)$$

Trong đó $\hat{x}_{k-1}$ là giá trị tín hiệu đã được lọc ở khung hình trước.

Bước 2: Lọc thông thấp vận tốc thô dx_k bằng hệ số lọc đạo hàm α_d để thu được

vận tốc mượt mà edx_k theo phương trình (2.15):

$$edx_k = \alpha_d \cdot dx_k + (1 - \alpha_d) \cdot edx_{k-1} \quad (2.15)$$

Hệ số lọc đạo hàm α_d phụ thuộc vào tần số cắt cố định $f_{c,d}$ (thường chọn mặc định bằng 1.0 Hz) được xác định theo phương trình (2.16):

$$\alpha_d = \frac{1}{1 + \frac{\tau_d}{dt}}, \quad \tau_d = \frac{1}{2\pi f_{c,d}} \quad (2.16)$$

Bước 3: Điều chỉnh tần số cắt thích nghi f_c của tín hiệu chính dựa trên độ lớn vận tốc mượt mà theo công thức (2.17):

$$f_c = f_{c,min} + \beta \cdot |edx_k| \quad (2.17)$$

Trong đó: $f_{c,min}$ là tần số cắt tối thiểu (Hz) giúp lọc bỏ tối đa nhiễu rung lắc (jitter) khi tay đứng yên ($|edx_k|$ giảm dần về 0); và β là hệ số nhạy tốc độ giúp tự động tăng tần số cắt f_c khi tay di chuyển nhanh ($|edx_k|$ lớn) để giảm thiểu độ trễ bám đuổi của tín hiệu lọc.

Bước 4: Tính hệ số lọc thích nghi α theo phương trình (2.18) và giá trị tín hiệu đầu ra sau lọc $\hat{x}_k$ theo phương trình (2.19):

$$\alpha = \frac{1}{1 + \frac{\tau}{dt}}, \quad \tau = \frac{1}{2\pi f_c} \quad (2.18)$$

$$\hat{x}_k = \alpha \cdot x_k + (1 - \alpha) \cdot \hat{x}_{k-1} \quad (2.19)$$

Việc áp dụng One Euro Filter cho cả hai trục tọa độ (x, y) giúp giảm rung lắc cho vị trí lưỡi dao khi ngón trỏ của người dùng đứng yên, đồng thời vẫn duy trì khả năng bám theo chuyển động nhanh ở mức phù hợp trong quá trình thử nghiệm.

2.6 Các thuật toán hình học mô phỏng 3D trong Unity

Để hiện thực hóa việc mô phỏng xương bàn tay 3D động học mượt mà và trực quan, hệ thống tích hợp hai thuật toán hình học quan trọng tại môi trường kết xuất đồ họa Unity:

2.6.1 Thuật toán khuếch đại tầm với thích nghi (Reach Remap)

Khi chuyển đổi vị trí bàn tay từ tọa độ camera thực tế sang không gian thế giới ảo (World Space) trong Unity, nếu sử dụng tỉ lệ ánh xạ tuyến tính 1:1, người dùng sẽ phải di chuyển bàn tay cơ học trên một không gian rất rộng để chạm tới các vật thể ở rìa xa. Thuật toán Reach Remap giải quyết vấn đề này bằng cách chuẩn hóa

tọa độ thô của cổ tay từ camera thực tế về khoảng tỷ lệ $[0, 1]$, sau đó ánh xạ phi tuyến vào vùng hoạt động ảo có kích thước tùy chỉnh.

Cho vị trí thô của cổ tay nhận được từ UDP là $P_{raw} = (x_{raw}, y_{raw}, z_{raw})$, kích thước khung hình camera thật là $W \times H$. Tọa độ chuẩn hóa độc lập với độ phân giải được tính bằng phương trình (2.20):

$$n_x = \frac{x_{raw}}{W}, \quad n_y = \frac{y_{raw}}{H} \quad (2.20)$$

Gọi vùng thể giới ảo mục tiêu có tâm hình học là $reachCenter = (c_x, c_y, c_z)$, phạm vi quét theo trục ngang và trục dọc tương ứng là $reachRange = (r_x, r_y)$. Tọa độ thể giới thực tế của điểm cổ tay tay ảo $wristWorld = (X_{unity}, Y_{unity}, Z_{unity})$ được xác định thông qua hệ số khuếch đại tầm với bằng phương trình (2.21):

$$\begin{aligned} X_{unity} &= c_x + (0.5 - n_x) \cdot r_x \\ Y_{unity} &= c_y + (n_y - 0.5) \cdot r_y \\ Z_{unity} &= c_z + \frac{z_{raw}}{D} \cdot G_z \end{aligned} \quad (2.21)$$

Trong đó: D là hệ số chia kích thước bàn tay ảo (coordinateDivisor); G_z là hệ số tăng cường độ sâu trục Z (depthGain). Công thức này đảm bảo bàn tay ảo phản chiếu chính xác chuyển động và giữ được tỷ lệ hình học tự nhiên so với các khớp xương xung quanh.

2.6.2 Thuật toán xác định trọng tâm và định hướng lòng bàn tay

Để đồng bộ vị trí của vật thể được cầm nắm khớp với lòng bàn tay của mô hình 3D, hệ thống cần tính toán chính xác trọng tâm (centroid) và hướng xoay (orientation) của lòng bàn tay dựa trên 5 điểm mốc chính: cổ tay P_0 (Wrist), gốc ngón trỏ P_5 (Index MCP), gốc ngón giữa P_9 (Middle MCP), gốc ngón áp út P_{13} (Ring MCP) và gốc ngón út P_{17} (Pinky MCP).

Tọa độ trọng tâm lòng bàn tay C_{palm} biểu thị vị trí đặt tâm của vật thể được tính bằng trung bình cộng tọa độ 5 điểm mốc trên qua công thức (2.22):

$$C_{palm} = \frac{1}{5} \sum_{i \in \{0,5,9,13,17\}} P_i \quad (2.22)$$

Định hướng không gian của lòng bàn tay được xác định bằng một hệ tọa độ trục giao địa phương xây dựng từ hai vectơ hướng. Vectơ chỉ hướng ngón tay $\vec{v}_{finger}$ (từ cổ tay đến gốc ngón giữa) và vectơ chỉ hướng ngang $\vec{v}_{side}$ (từ cổ tay đến gốc ngón

trở) được tính qua công thức (2.23):

$$\vec{v}_{finger} = P_9 - P_0, \quad \vec{v}_{side} = P_5 - P_0 \quad (2.23)$$

Vectơ pháp tuyến của lòng bàn tay $\vec{v}_{normal}$ biểu thị hướng mặt của lòng bàn tay hướng ra ngoài được xác định bằng tích có hướng (Cross Product) giữa hai vectơ trên theo phương trình (2.24):

$$\vec{v}_{normal} = \vec{v}_{finger} \times \vec{v}_{side} \quad (2.24)$$

Sau khi chuẩn hóa các vectơ chỉ hướng về dạng đơn vị $\hat{v}_{finger}$ và $\hat{v}_{normal}$, Unity sử dụng hàm Look Rotation để ánh xạ ma trận xoay này vào trục quay của mô hình bàn tay ảo và vật thể đang cầm nắm, tạo nên tương tác đồng bộ và chính xác.

Kết chương

Chương 2 đã trình bày chi tiết toàn bộ các cơ sở lý thuyết toán học và giải thuật được sử dụng trong hệ thống, bao gồm OpenCV tiền xử lý ảnh, MediaPipe Hands trích xuất điểm mốc, các phương trình chuẩn hóa hình học bàn tay, kiến trúc mô hình học máy MLP phân loại cử chỉ tĩnh/động, truyền thông mạng UDP, bộ lọc thích nghi chống rung One Euro Filter và thuật toán hình học biến đổi tọa độ mô phỏng 3D trong Unity. Trên cơ sở lý thuyết này, Chương 3 tiếp theo sẽ trình bày chi tiết về thiết kế kiến trúc hệ thống, cài đặt mã nguồn phần mềm thực tế và đánh giá các kết quả thực nghiệm đạt được.

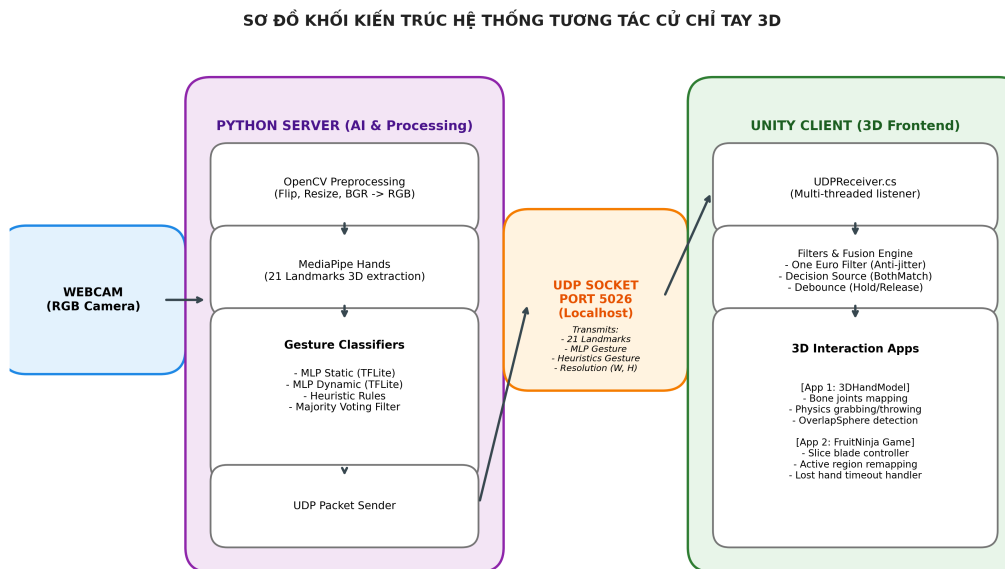
CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

Tổng quan chương

Chương 2 đã thiết lập các cơ sở lý thuyết toán học và giải thuật vững chắc cho hệ thống. Chương 3 này sẽ tập trung mô tả chi tiết quá trình hiện thực hóa kiến trúc phần mềm, lập trình các kịch bản tương tác trong Unity và đánh giá các kết quả thực nghiệm thu được. Nội dung chương bao gồm bốn phần chính: (i) Thiết kế kiến trúc hệ thống và giao thức truyền dữ liệu; (ii) Cài đặt module xử lý Python backend; (iii) Lập trình các ứng dụng tương tác 3D frontend trong Unity; và (iv) Đánh giá hiệu năng hệ thống về độ chính xác phân loại cử chỉ, tốc độ xử lý FPS, độ trễ hệ thống và hiệu quả lọc nhiễu của bộ lọc One Euro Filter.

3.1 Kiến trúc hệ thống và giao thức truyền dữ liệu

Hệ thống tương tác cử chỉ tay thời gian thực được thiết kế theo kiến trúc Client-Server phân tán nhằm tối ưu hóa tải tính toán. Việc tách biệt luồng xử lý AI (phía Python Server) và luồng kết xuất đồ họa mô phỏng 3D (phía Unity Client) giúp hệ thống duy trì hiệu năng cao trên các cấu hình máy tính cá nhân thông thường. Sơ đồ khối kiến trúc hệ thống được thể hiện trong Hình 3.1.



Hình 3.1: Sơ đồ khối kiến trúc hệ thống tương tác cử chỉ tay thời gian thực

Giao thức truyền dữ liệu giữa Python và Unity được xây dựng trên nền tảng socket mạng UDP. Phía Python mã hóa dữ liệu thành một chuỗi ký tự UTF-8 phân tách bằng dấu phẩy và gửi đến địa chỉ mạng cục bộ 127.0.0.1 trên cổng 5026.

Cấu trúc gói tin dữ liệu được đóng gói theo phương trình (3.1) như sau:

$$\text{Data} = [x_0, y_0, z_0, \dots, x_{20}, y_{20}, z_{20}, \text{GestureName}, W, H, \text{RuleGestureName}] \quad (3.1)$$

Trong đó: (i) x_i, y_i, z_i ($i \in [0, 20]$) là tọa độ không gian 3D thô của 21 điểm landmark khớp bàn tay do MediaPipe trích xuất (với trục y được biến đổi ngược chiều để phù hợp với hệ tọa độ Unity); (ii) GestureName là nhãn cử chỉ tĩnh được nhận dạng bởi mô hình học máy (ví dụ: "Open", "Close", "Pointer"); (iii) W cùng H tương ứng là chiều rộng và chiều cao thực tế của khung hình camera đầu vào, được gửi kèm để phía Unity thực hiện chuẩn hóa tỉ lệ độc lập với độ phân giải; và (iv) RuleGestureName là nhãn dự đoán cử chỉ tĩnh dựa trên thuật toán luật hình học Heuristics, phục vụ cho cơ chế dung hợp và xác thực quyết định phía Unity.

Trong phiên bản hiện tại, gói UDP sử dụng cho Unity truyền tọa độ landmark, nhãn cử chỉ tĩnh (từ cả MLP và Heuristics) cùng kích thước khung hình W, H . Kết quả nhận dạng cử chỉ động được xử lý và đánh giá hoàn toàn ở phía Python, chưa được tích hợp trực tiếp vào gói dữ liệu điều khiển Unity.

Để truyền dữ liệu, phía Python chuyển mảng dữ liệu thành một chuỗi văn bản dạng danh sách đơn giản, ví dụ `"[x_0, y_0, z_0, ..., 'GestureName', W, H, 'RuleGestureName']"`, sau đó gửi đi dưới dạng mảng byte mã hóa UTF-8. Phía Unity Client sử dụng lớp `UDPReceive.cs` để nhận luồng byte này qua một luồng chạy nền độc lập. Khi dữ liệu được nhận, chuỗi ký tự được xử lý lại ở luồng chính `Update()` bằng cách loại bỏ các ký tự ngoặc vuông ở hai đầu, tách các phần tử bằng dấu phẩy và chuyển từng phần tử về kiểu dữ liệu tương ứng.

3.2 Hiện thực hóa module xử lý Python Backend

Phần mềm phía Python được triển khai thông qua file mã nguồn `main.py` tích hợp đầy đủ ba chế độ hoạt động:

1. **Chế độ Dự đoán (Predict Mode - Mode 0):** Chế độ hoạt động chính của hệ thống. Luồng webcam từ camera được OpenCV đọc liên tục. Nhằm tối ưu hóa tốc độ xử lý của MediaPipe, độ phân giải camera được giảm xuống mức 960×540 pixel. Tọa độ của 21 điểm landmark sau khi trích xuất được chuẩn hóa theo công thức (2.6) và tiếp tục được đưa vào mô hình `GestureClassifier` để nhận diện cử chỉ tĩnh. Đồng thời, lịch sử tọa độ ngón trỏ được tích lũy vào hàng đợi để đưa vào mô hình `DynamicClassifier` nhận diện cử chỉ động.
2. **Chế độ Thu thập cử chỉ tĩnh (Collect Static Mode - Mode 1):** Cho phép người dùng nhấn các phím tương ứng với mã nhãn cử chỉ để thu thập dữ

liệu landmark chuẩn hóa tương ứng với các lớp cử chỉ (sử dụng 6 lớp tĩnh: Open, Close, Pointer, OK, Peace, ThumbsUp) và ghi trực tiếp vào tệp `gesture_classification.csv`.

3. **Chế độ Thu thập cử chỉ động (Collect Dynamic Mode - Mode 2):** Thu thập chuỗi hành trình di chuyển của ngón trỏ và lưu vào tệp dữ liệu `dynamic_gesture.csv` nhằm phục vụ cho quá trình huấn luyện mô hình cử chỉ động.

3.3 Lập trình tích hợp tương tác 3D trong Unity

Phía Unity nhận dữ liệu truyền từ Python thông qua lớp `UDPReceive.cs` chạy trên một luồng nền riêng biệt (background thread) để tránh gây nghẽn luồng kết xuất đồ họa chính. Hệ thống được triển khai trên hai ứng dụng độc lập:

3.3.1 Ứng dụng tương tác vật lý `3DHandModel`

Ứng dụng này ánh xạ tọa độ 21 điểm nhận được từ Server để tái tạo bộ khung xương bàn tay ảo và điều khiển các hoạt động tương tác. Phép chuyển đổi hệ tọa độ được kích bản `HandTracking.cs` thực hiện thông qua hai chế độ hoạt động chính:

- **Chế độ ánh xạ tỉ lệ 1:1 mặc định:** Tọa độ thô $x_{raw}, y_{raw}, z_{raw}$ nhận được từ Python được chuyển trực tiếp sang không gian thế giới (world unit) và co giãn qua hệ số chia `coordinateDivisor` (mặc định bằng 45). Hệ tọa độ được lật dọc (mirror) trên trục hoành để hiển thị tự nhiên qua biểu thức $x_{unity} = 7 - x_{raw}/divisor$, các trục còn lại tương ứng là $y_{unity} = y_{raw}/divisor$ và $z_{unity} = z_{raw}/divisor$.
- **Chế độ khuếch đại tầm với thích nghi (Reach Remap):** Khi kích hoạt `enableReachRemap`, hệ thống tách biệt hành vi di chuyển (tầm với) khỏi kích thước của bàn tay ảo. Tọa độ của điểm cổ tay (Wrist - điểm số 0) được chuẩn hóa về khoảng $[0, 1]$ dựa theo kích thước khung hình camera thật (W, H), sau đó ánh xạ phi tuyến vào vùng thế giới ảo rộng lớn xác định bởi tâm `reachCenter` và phạm vi quét `reachRange`. Tọa độ 20 điểm khớp còn lại được tính bằng khoảng lệch (offset) hình học tương đối so với cổ tay rồi chia cho hệ số `coordinateDivisor`. Cơ chế này giúp kích thước bàn tay ảo luôn giữ tỷ lệ tự nhiên cố định, đồng thời người dùng chỉ cần di chuyển tay nhẹ nhàng ngoài đời thực vẫn có thể tiếp cận và thao tác toàn bộ các vùng xa nhất của hộp chứa vật lý trong Unity.

Việc liên kết giữa các khớp xương được thực hiện tự động thông qua các đoạn thẳng trong thành phần `Line.cs` để tạo nên bộ khung xương trực quan. Đồng

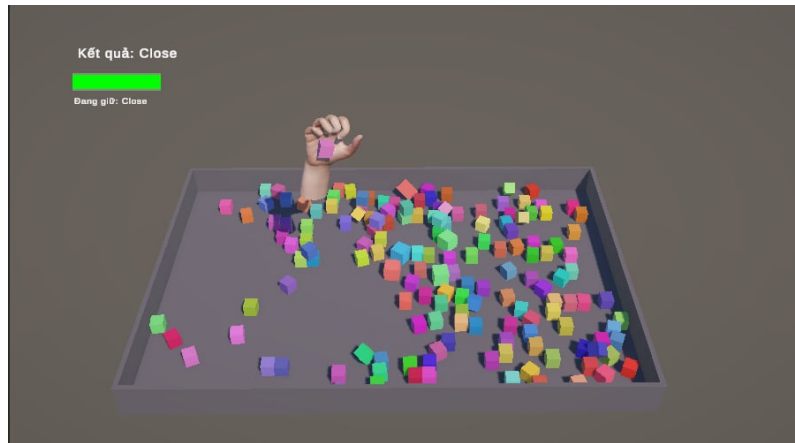
thời, module `GrabbableSpawner.cs` hỗ trợ cơ chế sinh tự động các khối hộp vật lý ngẫu nhiên hoặc theo vị trí xác định trước. Kịch bản hỗ trợ phương thức `SpawnBatch()` chạy trực tiếp tại runtime (Play Mode) thông qua nút Inspector Context Menu, cho phép liên tục sinh thêm các batch vật thể mới mà không cần reset trạng thái ứng dụng.

Để thực hiện tương tác cầm nắm, kịch bản `HandGrabber.cs` thực hiện ba chức năng chính: thứ nhất là xác định vị trí lòng bàn tay thông qua việc tính toán trọng tâm (centroid) và hướng xoay (rotation) của lòng bàn tay dựa trên 5 điểm mốc chính gồm cổ tay 0, gốc ngón trỏ 5, gốc ngón giữa 9, gốc ngón áp út 13 và gốc ngón út 17 (trong đó hướng lòng bàn tay được xác định bằng tích có hướng giữa hai vectơ ngón trỏ và ngón út); thứ hai là phát hiện va chạm bằng cách sử dụng các quả cầu kiểm tra va chạm vật lý (Overlap Sphere) với bán kính `pointRadius = 0.35` quanh các điểm mốc ngón tay để xác định các vật thể có thuộc tính `Grabbable` đang nằm trong tầm với; và thứ ba là thực hiện các hành vi cầm nắm và ném vật lý khi nhận được cử chỉ tính "Close" từ phía Python Server để gán vật thể làm con của lòng bàn tay (Grab), giải phóng vật thể khi cử chỉ chuyển trạng thái (Release), đồng thời áp dụng lực đẩy vật lý (`Rigidbody.velocity`) dựa trên vận tốc ném trung bình của 10 khung hình gần nhất để tạo cảm giác ném tự nhiên.

Để tối ưu hóa trải nghiệm tương tác vật lý và triệt tiêu các lỗi nhận dạng sai từ mô hình AI, ứng dụng tích hợp ba cơ chế nâng cao trực tiếp trong kịch bản điều khiển:

- **Dung hợp cử chỉ thích nghi (Gesture Source):** Cho phép lựa chọn giữa việc sử dụng độc lập mô hình MLP, thuật toán Heuristics, cơ chế dự phòng thích nghi (*EitherMatch*) hoặc cơ chế đồng thuận tuyệt đối (*BothMatch*). Chế độ *BothMatch* chỉ kích hoạt hành động cầm nắm khi cả mô hình MLP và thuật toán Heuristics đều đồng thời trả về nhãn "Close", giúp loại bỏ hoàn toàn các trường hợp vô tình kích hoạt cầm/ném do sai số của mô hình học máy.
- **Bộ lọc danh sách đen (Ignored Gestures):** Thiết lập danh sách các cử chỉ nhạy cảm dễ gây nhiễu để hệ thống bỏ qua và giữ nguyên trạng thái điều khiển trước đó khi nhận diện nhầm các nhãn này.
- **Cơ chế chống trễ và duy trì trạng thái (Grab/Release Debounce):** Cài đặt ngưỡng thời gian duy trì liên tục trạng thái cử chỉ trước khi chuyển đổi logic thực tế (0.15 giây đối với hành vi bắt đầu cầm nắm và 0.5 giây đối với hành vi thả rơi vật thể). Nhờ đó, nếu mô hình AI bị nhảy nhãn tức thời trong vài khung hình do tay người dùng chuyển động nhanh, vật thể vẫn được giữ chắc chắn mà không bị thả rơi tự do ngoài ý muốn. Giao diện mô phỏng bộ xương tay ảo

và quá trình cầm nắm tương tác vật lý này được biểu diễn trong Hình 3.2.



Hình 3.2: Giao diện mô phỏng xương bàn tay 3D và hành động cầm nắm vật thể ảo trong Unity

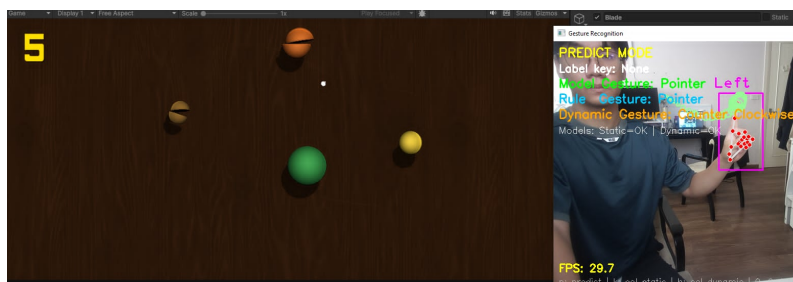
3.3.2 Ứng dụng trò chơi FruitNinja điều khiển bằng cử chỉ

Trò chơi FruitNinja được điều khiển bằng vị trí đầu ngón trỏ và trạng thái cử chỉ Pointer thay cho thao tác chuột truyền thống. Quá trình xử lý bao gồm hai nhiệm vụ chính: (i) ánh xạ tọa độ màn hình (`HandInput.cs`) bằng cách lấy duy nhất tọa độ của điểm đầu ngón trỏ (landmark 8), chuẩn hóa về khoảng $[0, 1]$ dựa trên kích thước camera, áp dụng bộ lọc thích nghi One Euro Filter với các tham số được sử dụng trong thử nghiệm là $\text{minCutoff} = 1.0$ và $\text{beta} = 0.4$ để giảm rung lắc (jitter) rồi remap ra kích thước màn hình game (`Screen.width`, `Screen.height`); và (ii) kích hoạt lưỡi dao (`Blade.cs`) khi nhận cử chỉ "Pointer" từ phía Python Server, lưỡi dao được kích hoạt và di chuyển theo vị trí ngón trỏ, đồng thời tạo vệt chém (`Trail Renderer`) và bật collider để cắt các quả bay lên khi vận tốc di chuyển vượt qua ngưỡng $\text{minSliceVelocity} = 0.01$.

Bên cạnh đó, nhằm tăng tính tiện dụng cho trò chơi thực tế, module `HandInput.cs` tích hợp thêm hai giải thuật tối ưu hóa động học:

- **Ánh xạ vùng hoạt động giới hạn (Active Region Remapping):** Thực hiện ánh xạ phi tuyến một phân vùng tọa độ thô thu hẹp từ camera (ví dụ khoảng $[0.3, 0.7]$) ra toàn bộ độ phân giải màn hình game thông qua hàm nội suy Remap. Cơ chế này giúp người dùng chỉ cần di chuyển tay trong một phạm vi nhỏ trước camera vẫn có thể quét hết các góc màn hình, giảm thiểu đáng kể sự mệt mỏi vật lý khi tương tác lâu.
- **Tự động xử lý mất dấu tay (Lost Hand Timeout):** Thiết lập bộ giám sát thời gian trễ 0.2 giây. Khi không nhận được gói tin UDP hợp lệ từ Server vượt quá ngưỡng này, hệ thống tự động giải phóng lưỡi dao và đưa trạng thái cử chỉ về

mặc định, tránh việc lưỡi dao bị đứng đờ tại vị trí biên cuối cùng khi người dùng đưa tay ra khỏi khung hình camera. Giao diện trực quan của trò chơi FruitNinja khi trải nghiệm điều khiển bằng ngón trỏ (cử chỉ Pointer) được thể hiện ở Hình 3.3.



Hình 3.3: Giao diện trò chơi FruitNinja điều khiển bằng cử chỉ ngón trỏ (Pointer)

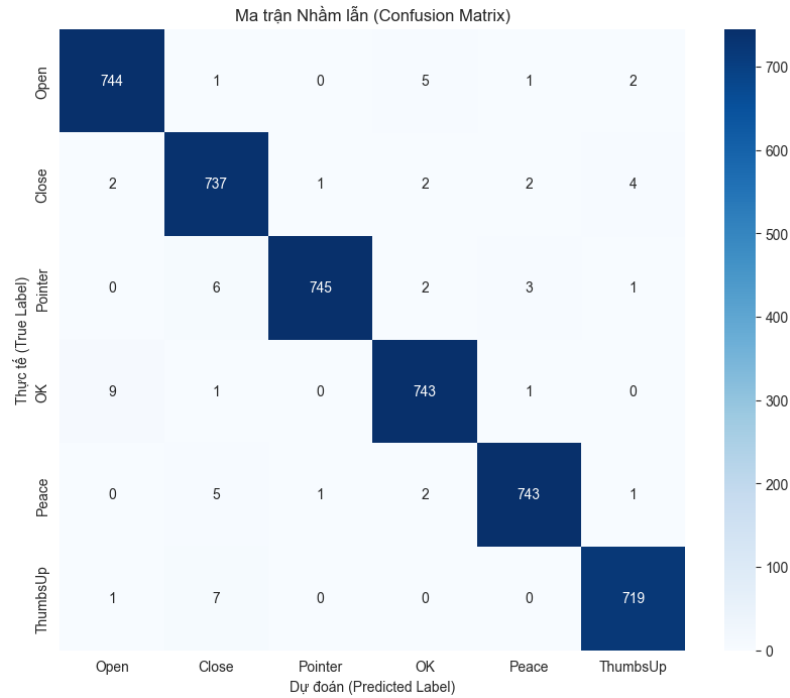
3.4 Đánh giá kết quả thực nghiệm

Quá trình kiểm thử hệ thống được thực hiện trên máy tính cấu hình văn phòng phổ thông sử dụng CPU Intel Core i5 và webcam tích hợp 720p.

3.4.1 Kết quả huấn luyện và Độ chính xác mô hình học máy

Hiệu năng nhận dạng của hệ thống phụ thuộc hoàn toàn vào kết quả huấn luyện của hai mô hình mạng nơ-ron nhân tạo MLP trên tập dữ liệu cử chỉ tĩnh và cử chỉ động thực nghiệm.

Đối với mô hình phân loại cử chỉ tĩnh sử dụng mạng nơ-ron MLP, tập dữ liệu thực tế thu thập gồm 13.471 mẫu huấn luyện và 4.491 mẫu kiểm thử phân tầng (stratified test set). Cấu trúc mạng MLP gồm lớp đầu vào 42 đặc trưng tọa độ landmark chuẩn hóa, hai lớp ẩn có kích thước lần lượt là 64 nơ-ron (kèm dropout 0.3) và 32 nơ-ron, lớp đầu ra sử dụng hàm Softmax dự đoán 6 lớp cử chỉ (Open, Close, Pointer, OK, Peace, ThumbsUp). Quá trình huấn luyện sử dụng bộ tối ưu hóa Adam với tốc độ học 0.001, chạy tối đa 500 epochs và dừng sớm ở epoch 104 do hàm val_loss không tiếp tục cải thiện. Bộ trọng số tối ưu nhất được khôi phục tại epoch 84 với val_loss đạt 0.0431 và độ chính xác val_accuracy đạt 98.78%. Khi đánh giá trên tập kiểm thử độc lập, mô hình đạt độ chính xác toàn cục (Accuracy) khoảng 99% trên tập kiểm thử. Kết quả dự đoán chi tiết đối với từng lớp cử chỉ tĩnh của mô hình MLP được thể hiện qua ma trận nhầm lẫn ở Hình 3.4 và báo cáo kết quả chi tiết ở Bảng 3.1.

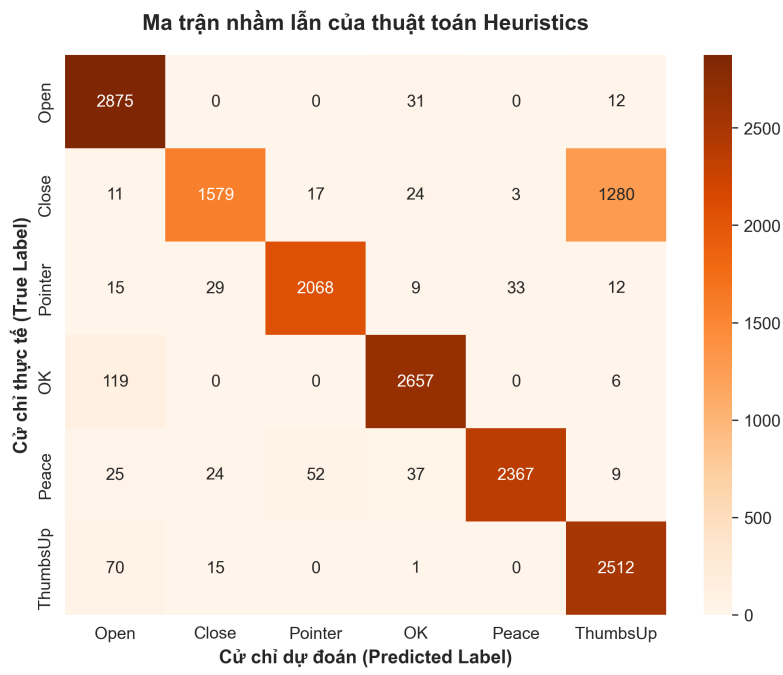


Hình 3.4: Ma trận nhầm lẫn của mô hình phân loại cử chỉ tĩnh bằng mạng MLP

Lớp cử chỉ tĩnh (MLP)	Precision	Recall	F1-Score	Support
Open	0.98	0.99	0.99	753
Close	0.97	0.99	0.98	748
Pointer	1.00	0.98	0.99	757
OK	0.99	0.99	0.99	754
Peace	0.99	0.99	0.99	752
ThumbsUp	0.99	0.99	0.99	727
Đoán đúng toàn cục (Accuracy)	0.99			4.491

Bảng 3.1: Báo cáo kết quả phân loại chi tiết (Classification Report) của mô hình cử chỉ tĩnh MLP

Để làm nổi bật ưu thế của việc sử dụng mạng nơ-ron, hệ thống tiến hành đánh giá đối chứng thuật toán hình học Heuristics (định nghĩa trong tệp mã nguồn `rule_gesture.py`) trên toàn bộ tập dữ liệu cử chỉ tĩnh gồm 17.962 mẫu. Kết quả thực nghiệm cho thấy thuật toán Heuristics chỉ đạt độ chính xác toàn cục (Accuracy) là 78.27%. Trong đó, lớp cử chỉ Close có độ phủ (Recall) thấp nhất là 53% với 1.280 lần bị phân loại nhầm thành ThumbsUp do ngón cái gấp lại vẫn bị công thức khoảng cách cứng nhắc nhận diện nhầm là duỗi. Lớp cử chỉ Pointer đạt F1-score là 80% do nhiều góc xoay tay bị thuật toán nhận diện nhầm thành trạng thái không xác định (None). Kết quả dự đoán chi tiết của thuật toán Heuristics được thể hiện qua ma trận nhầm lẫn ở Hình 3.5 và chi tiết chỉ số từng lớp ở Bảng 3.2.



Hình 3.5: Ma trận nhầm lẫn của thuật toán phân loại cử chỉ tĩnh bằng luật Heuristics

Lớp cử chỉ tĩnh (Heuristics)	Precision	Recall	F1-Score	Support
Open	0.92	0.95	0.94	3.011
Close	0.96	0.53	0.68	2.991
Pointer	0.97	0.68	0.80	3.027
OK	0.96	0.88	0.92	3.015
Peace	0.99	0.79	0.87	3.008
ThumbsUp	0.66	0.86	0.75	2.910
Đoán đúng toàn cục (Accuracy)	0.78			17.962

Bảng 3.2: Báo cáo kết quả phân loại chi tiết (Classification Report) của thuật toán hình học Heuristics

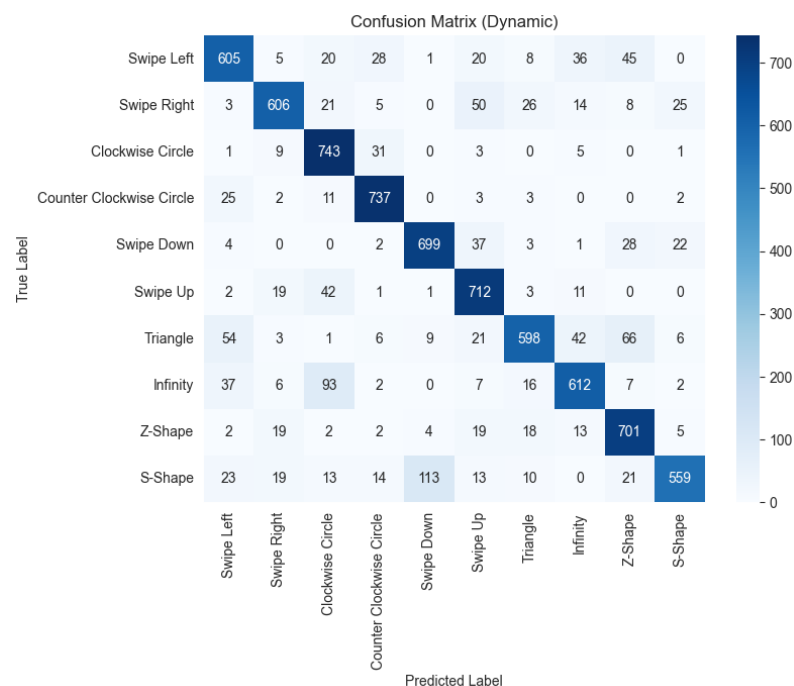
Để có cái nhìn tổng quan về cả hai giải thuật phân loại cử chỉ tĩnh, Bảng 3.3 tổng hợp các đặc tính hiệu năng chính của hai phương pháp:

Chỉ số so sánh (Cử chỉ tĩnh)	Mô hình học máy MLP	Thuật toán hình học Heuristics
Độ chính xác toàn cục (Accuracy)	99.0%	78.27%
F1-score trung bình (Macro Avg)	0.99	0.83
Lớp nhận diện tốt nhất (F1-score)	Open/Pointer/OK/Peace/ThumbsUp (0.99)	Open (0.94) / OK (0.92)
Lớp nhận diện kém nhất (F1-score)	Close (0.98)	Close (0.68) / ThumbsUp (0.75)
Thời gian huấn luyện mô hình	10 giây (84 epochs)	Không cần huấn luyện
Khả năng mở rộng cử chỉ mới	Rất cao (chỉ cần thêm nhãn và mẫu)	Thấp (phải thiết kế lại toàn bộ quy tắc)

Bảng 3.3: Bảng so sánh hiệu năng giữa mô hình học máy MLP và thuật toán hình học Heuristics

Đối với mô hình phân loại cử chỉ động dựa trên quỹ đạo, tập dữ liệu thu thập gồm 23.538 chuỗi quỹ đạo huấn luyện và 7.847 chuỗi kiểm thử thuộc 10 lớp cử

chỉ động chuyển động. Mạng MLP cho cử chỉ động nhận đầu vào là vectơ 64 chiều (chuỗi tọa độ 32 khung hình gần nhất của ngón trỏ), đi qua lớp Dropout tỉ lệ 0.2, lớp ẩn 128 nơ-ron (kèm dropout 0.4), lớp ẩn 64 nơ-ron và lớp đầu ra Softmax 10 lớp. Mô hình được thiết lập huấn luyện tối đa 1.000 epochs, thực hiện dừng sớm ở epoch 109 và khôi phục bộ trọng số tốt nhất ở epoch 89 với val_loss đạt 0.4421 và val_accuracy đạt 86.12%. Đánh giá trên tập kiểm thử độc lập 7.847 mẫu cho thấy mô hình đạt độ chính xác trung bình khoảng 84.0% trên tập kiểm thử độc lập. Ma trận nhầm lẫn ở Hình 3.6 thể hiện chi tiết kết quả phân loại, giúp quan sát các lớp dễ nhầm lẫn. Các chỉ số đánh giá chi tiết (Precision, Recall, F1-Score và Support) của từng lớp cử chỉ động được tổng hợp đầy đủ trong Bảng 3.4.



Hình 3.6: Ma trận nhầm lẫn của mô hình phân loại cử chỉ động

Lớp cử chỉ động	Precision	Recall	F1-Score	Support
Swipe Left	0.80	0.79	0.79	768
Swipe Right	0.88	0.80	0.84	758
Clockwise Circle	0.79	0.94	0.85	793
Counter Clockwise Circle	0.89	0.94	0.91	783
Swipe Down	0.85	0.88	0.86	796
Swipe Up	0.80	0.90	0.85	791
Triangle	0.87	0.74	0.80	806
Infinity	0.83	0.78	0.81	782
Z-Shape	0.80	0.89	0.84	785
S-Shape	0.90	0.71	0.79	785
Độ chính xác toàn cục (Accuracy)	0.84			7.847

Bảng 3.4: Báo cáo kết quả phân loại chi tiết (Classification Report) của mô hình cử chỉ động

3.4.2 Hiệu năng thời gian thực và Độ trễ hệ thống

Hệ thống được đánh giá hiệu năng thông qua hai chỉ số quan trọng là tốc độ xử lý khung hình (FPS) và tổng độ trễ phản hồi (latency). Về tốc độ xử lý (FPS), khi chạy luồng video gốc ở độ phân giải Full HD (1920×1080), MediaPipe gây quá tải cho CPU khiến tốc độ giảm xuống còn 18 FPS và xuất hiện hiện tượng giật lag rõ rệt; tuy nhiên, khi giảm độ phân giải xuống mức HD (960×540), tốc độ xử lý đã tăng lên và duy trì ổn định ở mức 30 - 32 FPS, đáp ứng yêu cầu cơ bản trong điều kiện thử nghiệm. Về độ trễ hệ thống, do Python và Unity chạy cục bộ trên cùng một máy tính, độ trễ truyền nhận qua socket UDP ở mức thấp trong quá trình thử nghiệm. Tổng độ trễ toàn trình, bao gồm thời gian camera thu ảnh, MediaPipe trích xuất khớp, Python suy luận cử chỉ và Unity cập nhật hiển thị, được ghi nhận dao động trong khoảng 35 – 45 ms. Mức trễ này chưa tạo ra hiện tượng gián đoạn rõ rệt khi quan sát và thao tác thử nghiệm.

3.4.3 Đánh giá hiệu quả làm mịn của bộ lọc One Euro Filter

Hiệu quả chống rung nhiễu của bộ lọc One Euro Filter được đánh giá thông qua quan sát độ ổn định của vị trí đầu ngón trỏ khi người dùng giữ tay tương đối cố định trước camera. Khi chưa áp dụng bộ lọc, tọa độ đầu ngón trỏ có thể dao động do nhiễu từ camera và sai số ước lượng landmark, khiến lưỡi dao trong game FruitNinja bị rung nhẹ. Khi áp dụng One Euro Filter với các tham số $f_{c,min} = 1.0$ Hz và $\beta = 0.4$, dao động tọa độ được giảm đáng kể, giúp chuyển động của lưỡi dao mượt hơn trong quá trình thử nghiệm. Tuy nhiên, hiệu quả của bộ lọc vẫn phụ thuộc vào điều kiện ánh sáng, chất lượng camera và tốc độ di chuyển tay của người dùng.

Kết chương

Chương 3 đã trình bày quá trình hiện thực hóa hệ thống từ việc đóng gói truyền dữ liệu qua UDP, thiết kế hệ xương tay ảo 3DHandModel và trò chơi tương tác FruitNinja, đến việc phân tích kết quả thực nghiệm cho thấy hệ thống đạt mức độ ổn định và độ chính xác phù hợp trong phạm vi thử nghiệm của đề tài. Tiếp theo, Chương 4 sẽ đưa ra những kết luận tổng kết và định hướng nghiên cứu phát triển tiếp theo của đề tài.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Tổng quan chương

Chương 3 đã trình bày chi tiết về kiến trúc phần mềm, hiện thực hóa các lớp xử lý và các kết quả thực nghiệm thực tiễn của đề tài. Chương 4 này sẽ đưa ra những đánh giá tổng kết cuối cùng về quá trình thực hiện bài tập lớn môn học Thực tập cơ sở. Nội dung chương bao gồm hai phần chính: (i) Phần Kết luận tổng hợp các kết quả đạt được, ưu điểm và mặt hạn chế còn tồn tại của hệ thống; và (ii) Phần Hướng phát triển vạch ra các định hướng nghiên cứu, nâng cấp giải pháp kỹ thuật trong tương lai.

4.1 Kết luận

Đề tài nghiên cứu và xây dựng hệ thống tương tác người - máy bằng cử chỉ tay thời gian thực dựa trên xử lý ảnh và học máy, kết hợp mô phỏng môi trường 3D đã hoàn thành các mục tiêu chính trong phạm vi bài tập lớn. Sau quá trình thiết kế, triển khai thực tế và thử nghiệm hiệu năng, một số kết quả chính đã đạt được bao gồm:

- 1. Xây dựng hệ thống tương tác thời gian thực phù hợp:** Hệ thống đạt tốc độ xử lý khoảng 30 - 32 FPS trong điều kiện thử nghiệm, cho phép phản hồi tương đối tốt đối với chuyển động và cử chỉ của người dùng. Tổng độ trễ toàn trình ước lượng trong điều kiện thử nghiệm cục bộ dao động từ 35 - 45 ms, tạo cảm giác tương tác bám đuổi tương đối mượt.
- 2. Phương pháp nhận dạng cử chỉ chính xác và linh hoạt:** Kết hợp hiệu quả giữa thuật toán luật hình học Heuristics (dựa trên quy tắc co/duỗi ngón tay) và phương pháp học máy có huấn luyện (sử dụng mạng nơ-ron nhân tạo dạng truyền thẳng MLP và triển khai dưới định dạng TensorFlow Lite). Mô hình phân loại cử chỉ tĩnh đạt độ chính xác khoảng 99% trên tập kiểm thử, cho thấy khả năng nhận dạng ổn định đối với các trạng thái bàn tay cơ bản. Mô hình phân loại cử chỉ động đạt độ chính xác khoảng 84% trên tập kiểm thử, phản ánh khả năng nhận dạng bước đầu đối với các quỹ đạo chuyển động, tuy nhiên vẫn còn nhầm lẫn ở một số lớp có hình dạng tương đồng.
- 3. Đánh giá đối chứng thực nghiệm rõ ràng:** Kết quả so sánh trực tiếp chỉ ra mô hình học máy MLP vượt trội rõ rệt so với thuật toán Heuristics về độ chính xác toàn cục (99.0% so với 78.27%), F1-score và khả năng mở rộng cử chỉ mới, trong khi Heuristics chỉ giữ lợi thế về việc không tốn thời gian huấn luyện và cấu hình tối giản.

4. **Hiện thực hóa các ứng dụng tương tác 3D trực quan:** Thiết lập cơ chế truyền dữ liệu cục bộ qua giao thức UDP để kết nối module Python AI với môi trường Unity. Xây dựng hệ xương bàn tay ảo mô phỏng 3DHandModel tích hợp cơ chế cầm nắm, ném vật thể ảo dựa trên mô phỏng vật lý và trò chơi tương tác giải trí FruitNinja điều khiển bằng ngón trỏ.
5. **Giảm hiện tượng rung giật và nâng cao độ ổn định tương tác:** Áp dụng thành công thuật toán bộ lọc thông thấp thích nghi One Euro Filter trong Fruit Ninja kết hợp với cơ chế giới hạn vùng hoạt động (*Active Region Remapping*) giúp giảm thiểu tối đa hiện tượng rung lắc tọa độ của đầu ngón tay khi tương tác. Đồng thời, việc cài đặt giải thuật dung hợp cử chỉ (*BothMatch*) và trễ chống nhiễu cầm nắm (*Debounce*) đã giảm đáng kể tình trạng nhảy nhảm lẫn tức thời của mô hình AI, giúp trải nghiệm tương tác vật lý (cầm/nắm/ném) trong ứng dụng mô phỏng xương bàn tay đạt độ ổn định và chính xác cao trong thực tế.

Bên cạnh những kết quả tích cực đã đạt được, hệ thống vẫn tồn tại một số hạn chế kỹ thuật cần được khắc phục: (i) độ chính xác trích xuất khớp của MediaPipe phụ thuộc nhiều vào điều kiện ánh sáng xung quanh, dễ bị nhảy nhiều hoặc mất dấu vết (lost tracking) khi ánh sáng yếu hoặc ngược sáng; (ii) hiện tượng tự che khuất (self-occlusion) khi bàn tay xoay nghiêng hoặc các ngón tay gấp khuất sau lòng bàn tay làm MediaPipe dự đoán sai tọa độ khớp, dẫn đến việc hiển thị tay ảo trong Unity bị méo lệch; và (iii) phân loại cử chỉ tĩnh đôi khi bị nhầm lẫn giữa các lớp có cấu trúc gần giống nhau khi người dùng di chuyển tay quá nhanh giữa các trạng thái.

4.2 Hướng phát triển

Để nâng cao độ ổn định, độ chính xác và mở rộng khả năng ứng dụng thực tiễn của hệ thống, hướng nghiên cứu phát triển tiếp theo của đề tài sẽ tập trung vào các công việc sau:

1. **Nâng cấp mô hình phân loại cử chỉ động:** Do mô hình cử chỉ động hiện tại sử dụng MLP trên vector quỹ đạo đã làm phẳng nên khả năng khai thác quan hệ thời gian còn hạn chế. Trong tương lai, có thể nghiên cứu các mô hình chuỗi thời gian như LSTM, GRU hoặc Transformer để học tốt hơn đặc trưng động học của cử chỉ.
2. **Hỗ trợ tương tác hai bàn tay và đa cử chỉ phối hợp:** Mở rộng hệ thống để trích xuất và nhận diện đồng thời cử chỉ của cả hai tay. Thiết kế các cử chỉ phối hợp nâng cao như zoom vật thể bằng hai tay, xoay đối tượng 3D hoặc tương tác giao diện người dùng (UI) hai tay để nâng cao số bậc tự do nhập liệu.

3. **Mở rộng hệ thống tương tác hai bàn tay ở góc nhìn thứ nhất (First-Person):** Trích xuất và xử lý đồng thời 21 điểm landmark của cả hai bàn tay từ Python truyền qua giao thức UDP để điều khiển trực tiếp cặp tay ảo trong môi trường game 3D. Nghiên cứu và áp dụng cơ chế vật lý ragdoll, các ràng buộc khớp (joint limits) và vùng va chạm ảo (colliders) độc lập cho từng bàn tay. Hệ thống cho phép người chơi sử dụng cả hai tay phối hợp thực hiện các thao tác phức tạp như cầm, nắm, ném vật thể, điều khiển vũ khí (súng, kiếm) hoặc giải các câu đố đòi hỏi sự chuyển động linh hoạt của từng ngón tay. Giải pháp này hướng tới việc tái hiện trải nghiệm tương tác không gian tương tự các ứng dụng VR/AR chuyên dụng nhưng chạy trên nền tảng webcam RGB thông thường, giúp tăng khả năng tiếp cận cho người dùng.
4. **Ứng dụng luồng dữ liệu cử chỉ và tọa độ ngón tay vào không gian game 2D:** Ánh xạ luồng dữ liệu 21 điểm landmark từ Python vào hệ tọa độ camera Orthographic hoặc Canvas UI trong không gian 2D. Hệ thống tập trung bắt các cử chỉ tĩnh/động kết hợp trích xuất vị trí độc lập của từng đầu ngón tay để tạo ra các tính năng gameplay độc đáo như: vẽ quỹ đạo để đại pháp thuật thi triển (cast spell), thay đổi trạng thái co/duỗi các ngón tay để tạo hình và xoay các khối gạch giải đố (giống game Block Blast), hoặc tận dụng khoảng cách và vị trí linh hoạt giữa các ngón tay để làm tính năng ngắm bắn, bóp cò, tương tác vi mô với môi trường 2D.

Kết chương

Chương 4 đã tổng kết các kết quả đạt được và ý nghĩa thực tiễn của đề tài bài tập lớn Thực tập cơ sở, đồng thời phân tích các ưu điểm cùng những hạn chế kỹ thuật hiện tại của hệ thống tương tác cử chỉ tay thời gian thực. Các hướng phát triển được đề xuất là cơ sở quan trọng để tiếp tục nâng cấp hệ thống trong tương lai, hướng tới một giải pháp tương tác người - máy tự nhiên, mượt mà và tiện dụng hơn.

TÀI LIỆU THAM KHẢO

- [1] J. Ravikiran, K. Mahesh, S. Mahishi, et al., “Hand gesture recognition for sign language transcription,” *International Journal of Computer Applications*, vol. 78, no. 12, 2013.
- [2] G. Bradski, “The opencv library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [3] C. Lugaresi et al., “Mediapipe: A framework for perceiving and processing reality,” *arXiv preprint arXiv:1906.08172*, 2019.
- [4] Google LLC, *TensorFlow Lite: Machine learning on mobile and embedded devices*, <https://www.tensorflow.org/lite>, 2017.
- [5] Unity Technologies, *Unity Real-Time Development Platform*, <https://unity.com>, Accessed: May 2026, 2026.
- [6] J. Postel, “User datagram protocol,” IETF, RFC 768, 1980.
- [7] G. Casiez, N. Roussel, and D. Vogel, “One euro filter: A simple 1-parameter adaptive filter for noisy input in applications,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ACM, 2012, pp. 2527–2530.